

Graph Theory Guided Reinforcement Learning for Network Design with High Algebraic Connectivity under Vertex and Edge Constraints

Zeru Zhu Marco Gamarra Ji Liu

Abstract—This paper studies an open network design problem of constructing graph topologies with maximum algebraic connectivity under fixed numbers of vertices and edges. A density-dependent approach is proposed that decomposes the design task into two stages: a high-quality, low-cost structural backbone construction and a reinforcement learning completion stage. The method selects the backbone based on density, using complete multipartite graphs in dense regimes and Cayley graphs in low-to-intermediate regimes, and then refines the remaining edges through a learning-based procedure. Numerical results show that the proposed method achieves high algebraic connectivity with favorable runtime scalability, outperforming existing methods. These results support a structural-family-first strategy for scalable network design.

I. INTRODUCTION

Over the past two decades, consensus has been extensively studied and has achieved notable success [1]–[3], finding applications in a broad range of distributed control and computation problems [4]–[7]. Consensus processes are fundamental to almost all cooperative multi-agent algorithms and, in many cases, significantly influence or even determine their convergence rate.

A typical continuous-time linear consensus process among n agents over a connected simple graph G can be modeled by a linear differential equation of the form $\dot{x}(t) = -Lx(t)$, where $x(t)$ is the state vector in $\mathbb{R}^n$ and L is the “Laplacian matrix” of G . The i th entry of $x(t)$ represents the state of agent i . The multi-agent system is said to reach a consensus if all $x_i(t)$ are equal. For a simple graph G with n vertices, its Laplacian matrix is defined as $L = D - A$, where D and A denote its degree matrix and adjacency matrix, respectively. Here, D is an $n \times n$ diagonal matrix whose i th diagonal entry equals the degree of vertex i , and A is an $n \times n$ matrix whose i,j th entry equals 1 if (i, j) is an edge in G and otherwise equals 0. It is well known that L is positive semidefinite with smallest eigenvalue 0, and its second smallest eigenvalue, called the algebraic connectivity of G , is positive if and only if G is connected [8]. It has been shown that $\dot{x}(t) = -Lx(t)$ will reach a consensus if and only if G is connected. More importantly, the (worst-case) convergence rate of consensus is characterized by the algebraic connectivity, with larger values leading to faster consensus [2].

Motivated by the above discussion, a natural and fundamental problem is to design network topologies that achieve

fast, or sometimes fastest, consensus. This problem has been extensively studied for a long time, among others [9]–[14]. Despite these efforts, the following question remains largely open: Given fixed numbers of vertices and edges, which graphs maximize the algebraic connectivity? To date, this question has only been theoretically answered in certain special cases [14], [15]. Beyond the challenges in theoretical analysis, the question is also computationally intractable in numerical experiments, especially for large-scale graphs. Indeed, it leads to an NP-hard combinatorial optimization problem [16]. Even though exhaustive combinatorial search is conceptually possible, its super-exponential growth makes it computationally impractical beyond very small graph sizes. The structural characterization of optimal graphs that maximize algebraic connectivity therefore remains poorly understood, let alone the associated computational complexity.

A. Related Work and Benchmark Methods

Due to the theoretical and computational hurdles, existing literature includes practical heuristics (typically without guarantees) and exact formulations (typically with limited scalability), such as MISDP-based approaches. Constructing graphs from scratch under fixed vertex/edge budgets has received less attention: [17] is restricted to special regular graphs, and [18] studies a weighted MISDP formulation with outer-approximation procedures.

Most existing works investigate a closely related augmentation problem: given a connected graph, how to add a fixed number of edges to maximally increase algebraic connectivity. This problem is NP-hard [16]. Prior work includes SDP relaxations for Boolean edge-addition formulations [19], as well as MISDP formulations [20]. A number of heuristic and approximation methods have also been proposed. Notable among them are Fiedler vector based greedy methods [19], [21], secular equation (for the Laplacian under a rank-one perturbation) guided edge selection [22], tabu search [23], and Frank-Wolfe optimization with Fiedler vector gradients [24]. In principle, these methods can be extended to construct a graph from scratch via different strategies, e.g., applying them iteratively.

These methods are computationally expensive and do not scale well to large graph construction. We benchmark representative methods in [18]–[20], [23]–[25] for medium-sized graphs from scratch; results indicate that Fiedler vector based greedy (FVG) [19] gives the best practical quality-runtime trade-off among full-construction baselines (Section IV-B), so we adopt it as the primary benchmark. We do not benchmark [21] since it is a decentralized implementation

Z. Zhu is with the Department of Applied Mathematics and Statistics at Stony Brook University. M. Gamarra is with the Air Force Research Laboratory. J. Liu is with the Department of Electrical and Computer Engineering at Stony Brook University. Email addresses: zeru.zhu@stonybrook.edu, marco.gamarra@us.af.mil, ji.liu@stonybrook.edu.

of FVG under a different problem setting. We also exclude [22] in the main benchmark because it is an exact $\Delta\lambda_2$ greedy algorithm; in our setting it has higher cost and does not improve the quality-runtime trade-off relative to FVG.

For a graph, any eigenvector corresponding to the second smallest eigenvalue of its Laplacian matrix is called a Fiedler vector of the graph [8]. We apply the Fiedler vector based greedy method to construct a graph from scratch by iteratively adding edges that maximize the increase in algebraic connectivity. Specifically, at each iteration, the increase caused by adding an edge (i, j) is approximated via first-order eigenvalue perturbation as $(v_i - v_j)^2$, where v is a selected Fiedler vector of the current graph; if λ_2 has multiplicity greater than one, v is chosen from the corresponding eigenspace.

B. Problem Formulation

For a connected simple graph $G = (V, E)$ with $|V| = n$ and $|E| = m$, its Laplacian eigenvalues are ordered as $0 = \lambda_1(L(G)) \leq \lambda_2(L(G)) \leq \dots \leq \lambda_n(L(G))$. Here $\lambda_2(L(G))$ is the algebraic connectivity. Let $\mathcal{G}_n$ denote the set of all connected simple graphs with n vertices. For fixed (n, m) , define $\mathcal{G}_{n,m} = \{G \in \mathcal{G}_n : |E| = m\}$. The network design problem of interest is

$$\max_{G \in \mathcal{G}_{n,m}} \lambda_2(L(G)). \quad (1)$$

We refer to (1) as algebraic connectivity maximization. This is an NP-hard combinatorial optimization problem [16]. Because of this, the goal is to construct a graph with algebraic connectivity as large as possible for fixed n and m .

C. Contributions

This paper proposes a graph theory guided reinforcement learning approach to the algebraic connectivity maximization problem. The approach decomposes the network design into two stages. First, we develop a density-dependent, high-quality, low-cost backbone stage for fixed- (n, m) algebraic-connectivity maximization, using complete multipartite and Cayley graph families with routing and fallback (cf. Section II). Second, we formulate completion as a residual edge-allocation problem from the selected backbone and use reinforcement learning (RL) to optimize only this sequential residual stage (cf. Section III). Together, this backbone-plus-completion decomposition gives the best quality/runtime scaling among tested baselines as n increases, outperforms all existing methods, and thus supports a structural-family-first viewpoint for scalable algebraic-connectivity maximization (cf. Section IV).

II. DENSITY-DEPENDENT BACKBONE CONSTRUCTION

Most existing methods in related work assume a given initial graph and then optimize edge additions. Our approach treats initialization as a design variable that strongly affects both remaining-edge search difficulty and final graph quality. Meanwhile, simulations show that graphs with high algebraic connectivity exhibit distinct structures depending on graph

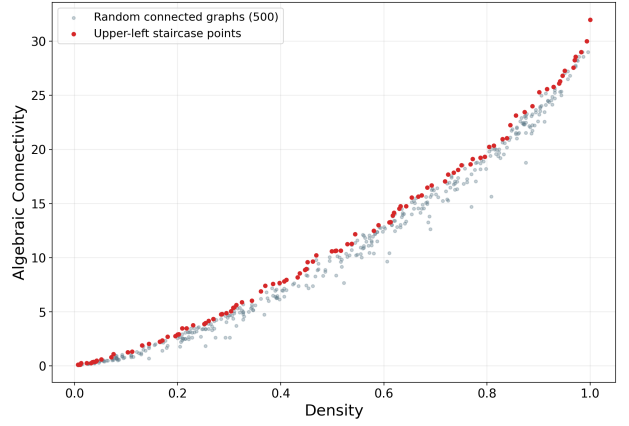


Fig. 1. Random connected graphs ($n = 32$); highlighted points are weak-maximal staircase points in (ρ, λ_2) .

density. This motivates a density-dependent structural initialization that chooses the initial graph family according to edge density. To parameterize edge budgets uniformly across different n , we use the connected-range density

$$\rho = \frac{m - (n - 1)}{\binom{n}{2} - (n - 1)}, \quad \text{with } n - 1 \leq m \leq \binom{n}{2}. \quad (2)$$

It is easy to see that $\rho \in [0, 1]$. $\rho = 0$ corresponds to the minimum connectivity budget ($m = n - 1$) and $\rho = 1$ corresponds to the complete graph. For each target (n, m) , we first construct a connected graph $G_0 = (V, E_0)$ with $|E_0| = m_0 \leq m$, aiming for large $\lambda_2(G_0)$ relative to its edge budget. The remaining $m - m_0$ edges are added in a later completion stage. We refer to this initialized graph G_0 as a *backbone*. Section II-A proposes the envelope backbone, Section II-B proposes the Cayley backbone, Section II-C presents an algorithm for selecting between the two backbones based on density (density routing), and Section II-D analyzes the algorithm complexity and scalability.

A. Envelope Backbone via Complete Multipartite Graphs

For a fixed node count n , each graph in $\mathcal{G}_n$ corresponds to a discrete point in the (ρ, λ_2) plane. Since λ_2 is monotone nondecreasing under edge addition, increasing edge density does not reduce the best achievable λ_2 ; hence, across the density spectrum, any maximizer of (1) for some feasible m must lie on the weak upper-left staircase, i.e., weak-maximal points under the order “smaller ρ , larger λ_2 .” See Fig. 1. We formalize this notion as follows.

Definition 1: Let $\mathcal{P} \subset \mathbb{R}^2$ be a finite point set, with points written as $p = (x_p, y_p)$. For $p, q \in \mathcal{P}$, we say that q *weakly dominates* p if $x_q \leq x_p$ and $y_q \geq y_p$. We say that q *strictly dominates* p if $x_q \leq x_p$, $y_q \geq y_p$, and at least one inequality is strict. A point p is *weak-maximal* (resp. *strict-maximal*) if no point in $\mathcal{P}$ weakly dominates (resp. strictly dominates) p .

In our setting, $x = \rho$ and $y = \lambda_2$, and $\mathcal{P}$ is the set of graph-induced points at fixed n . In this convention, weak dominance is strict in λ_2 and non-strict in ρ ; weak-maximality characterizes where optimizers of (1) can lie,

while strict-maximality keeps only the left boundary among points with equal λ_2 . Characterizing these maximal sets of $\mathcal{G}_n$ directly is generally intractable. We therefore study them within a structured subfamily where edge count and λ_2 admit closed-form expressions. We next introduce the complete multipartite family $\mathcal{K}_n$, which is known to contain graphs with strong algebraic connectivity [14], [15]. An illustrative subset of $\mathcal{K}_n$ in the (ρ, λ_2) plane is shown in Fig. 2.

Definition 2: Let $r \in \{2, \dots, n\}$ and $1 \leq n_1 \leq \dots \leq n_r$ with $\sum_{i=1}^r n_i = n$. The complete multipartite graph $K_{n_1, \dots, n_r}$ has all edges between different parts and no edges within any part. Denote by $\mathcal{K}_n \subseteq \mathcal{G}_n$ the family of all such complete multipartite graphs on n vertices.

It is known that when all parts have equal size (i.e., r divides n), this graph is a maximizer of λ_2 for the corresponding edge budget [26]. We refer to this case as a balanced complete multipartite graph.

The first ingredient is a closed-form edge-count identity.

Lemma 1: For $G = K_{n_1, \dots, n_r}$,

$$m(G) = \sum_{1 \leq i < j \leq r} n_i n_j = \frac{1}{2} \left(n^2 - \sum_{i=1}^r n_i^2 \right). \quad (3)$$

Proof: Edges appear only across different parts, so $m(G) = \sum_{1 \leq i < j \leq r} n_i n_j$. Using $(\sum_{i=1}^r n_i)^2 = \sum_{i=1}^r n_i^2 + 2 \sum_{1 \leq i < j \leq r} n_i n_j$ and $\sum_i n_i = n$, we obtain (3). ■

Hence edge minimization at fixed n is equivalent to maximizing $\sum_i n_i^2$.

The second ingredient is a closed-form expression for λ_2 in this family.

Lemma 2: Let $G = K_{n_1, \dots, n_r}$ with $2 \leq r \leq n-1$ and $n_1 \leq \dots \leq n_r$. Then

$$\lambda_2(G) = n - n_r. \quad (4)$$

For $r = n$, $G = K_n$ and $\lambda_2(G) = n$.

Proof: The lemma is a direct consequence of Proposition 3.2 in [27]. ■

This shows that, within $\mathcal{K}_n$, fixing $\lambda_2 = k$ is equivalent to fixing the largest part size as $n - k$.

We now define the envelope point at a fixed λ_2 -level.

Definition 3: Fix $k \in \{1, \dots, n-2\}$. Among all $G \in \mathcal{K}_n$ with $\lambda_2(G) = k$, a graph with minimum edge count is called an envelope point at level k .

By definition, these are exactly strict-maximal points of $\mathcal{K}_n$ in the (ρ, λ_2) plane. The next proposition gives an explicit construction and edge formula.

Proposition 1 (Envelope Point at Level k): Fix $k \in \{1, \dots, n-2\}$, and write

$$n = q(n - k) + b, \quad 0 \leq b < n - k. \quad (5)$$

An envelope point is obtained by taking q parts of size $n - k$, and, if $b > 0$, one additional part of size b . Its edge count is

$$m_{\text{env}}(k) = \frac{1}{2} (n^2 - q(n - k)^2 - b^2). \quad (6)$$

Proof: By Lemma 2, fixing $\lambda_2 = k$ is equivalent to fixing the largest part size as $n_r = n - k$. Thus feasible partitions satisfy $n_i \leq n - k$ and $\max_i n_i = n - k$. By Lemma 1, minimizing $m(G)$ is equivalent to maximizing $\sum_i n_i^2$.

In any maximizer of $\sum_i n_i^2$ under these constraints, at most one part size is strictly smaller than $n - k$. Indeed, suppose there are two indices $i \neq j$ with $1 \leq n_i \leq n_j < n - k$. Replacing $(n_i, n_j) \mapsto (n_i - 1, n_j + 1)$, where a zero part (if $n_i = 1$) is discarded, keeps feasibility and strictly increases $\sum_i n_i^2$, since $(n_i - 1)^2 + (n_j + 1)^2 - (n_i^2 + n_j^2) = 2(n_j - n_i) + 2 > 0$. This contradicts maximality. Therefore all but at most one parts must have size exactly $n - k$, giving the stated partition. Writing $n = q(n - k) + b$, $0 \leq b < n - k$, yields that form, and substituting into (3) gives (6). ■

Proposition 1 yields a closed-form staircase of strict-maximal points in $\mathcal{K}_n$, providing a cheap dense-regime backbone library. Detailed envelope-versus-baseline performance comparisons are deferred to Figures 5–7.

At very sparse densities, the first two envelope levels are key anchors: at $k = 1$, Proposition 1 gives $K_{1, n-1}$, i.e., the star graph, which is optimal at the minimum connected budget $m = n - 1$ [28]; at $k = 2$, it gives $K_{2, n-2}$, whose optimality at the corresponding budget is a well-known open conjecture [29].

For backbone construction, given target (n, m) , choose $k^* = \max\{k \in \{1, \dots, n-2\} : m_{\text{env}}(k) \leq m\}$; because $m_{\text{env}}(k)$ is monotone nondecreasing, k^* is found by integer binary search in $O(\log n)$ time. The envelope candidate is then the complete multipartite graph from Proposition 1 at level k^* (equivalently, using (5)–(6)). Its algebraic-connectivity score is $\lambda_2 = k^*$ (or $\lambda_2 = n$ when $m = \binom{n}{2}$), and this candidate is passed to the routing/selection stage in Section III-C. Algorithm 1 summarizes this deterministic envelope construction.

Algorithm 1 Envelope Backbone Construction

- 1: **Input:** n, m
 - 2: **Output:** Envelope backbone G_{env} with $|E(G_{\text{env}})| \leq m$
 - 3: **if** $m = \binom{n}{2}$ **then**
 - 4: **return** K_n with $\lambda_2 = n$
 - 5: **end if**
 - 6: $k^* \leftarrow \max\{k \in \{1, \dots, n-2\} : m_{\text{env}}(k) \leq m\}$ by integer binary search on (6)
 - 7: $q \leftarrow \lfloor n/(n - k^*) \rfloor$, $b \leftarrow n - q(n - k^*)$
 - 8: build $K_{n_1, \dots, n_r}$ with q parts of size $n - k^*$ and one part b if $b > 0$, $G_{\text{env}} \leftarrow K_{n_1, \dots, n_r}$
 - 9: **return** G_{env} and $\lambda_2(G_{\text{env}}) = k^*$
-

B. Cayley Backbone via Coset-Based Generating Sets

While envelope points from $\mathcal{K}_n$ are strong in dense regimes, our experiments show a clear weakness for $\rho < \frac{1}{2}$, where their λ_2 is non-competitive (see Figure 7). Therefore, the need for a second backbone family is driven not only by limited envelope coverage across feasible density levels, but also by quality degradation in low-to-intermediate densities.

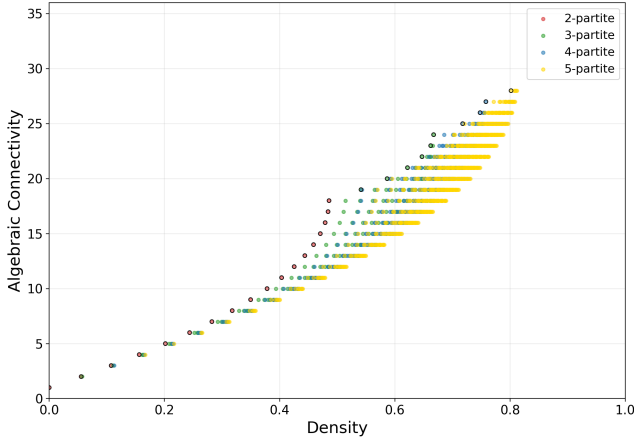


Fig. 2. Complete 2/3/4/5-partite graphs in $\mathcal{K}_{36}$ on the (ρ, λ_2) plane.

Balanced complete multipartite graphs are a natural anchor because they are guaranteed optimal at their corresponding budgets [26]. This raises a natural question: beyond $\mathcal{K}_n$, is there a structured graph family that extends balanced complete multipartite graphs, yet can realize stronger candidates in other density ranges? Cayley graphs provide such a generalization; as shown below, complete balanced multipartite graphs arise as a special Cayley case.

We next give a formal definition of Cayley graphs.

Definition 4: Let Γ be a finite group and $S \subseteq \Gamma \setminus \{e\}$ with $S = S^{-1}$. The undirected Cayley graph $\text{Cay}(\Gamma, S)$ has vertex set Γ and edge set $\{\{g, gs\} : g \in \Gamma, s \in S\}$.

The inverse-closed condition ensures undirected edges, while group actions provide strong symmetry and reusable structure for sampling.

The next proposition establishes that the balanced complete multipartite case is a special instance of Cayley graphs.

Proposition 2 (Balanced Multipartite as Cayley): Let Γ be a finite group, and let $H < \Gamma$ be a proper subgroup of index $r \geq 2$. Define $S = \Gamma \setminus H$. Then $\text{Cay}(\Gamma, S)$ is a complete balanced r -partite graph whose parts are the left cosets of H .

Proof: Since $e \in H$, we have $e \notin S$. Also $S^{-1} = S$, so $\text{Cay}(\Gamma, S)$ is an undirected Cayley graph. Let $\Gamma = \bigsqcup_{\ell=1}^r g_\ell H$ be the left-coset decomposition, where $\bigsqcup$ denotes disjoint union. Each part has size $|H|$, so the partition is balanced. For $x, y \in \Gamma$, if they are in the same coset then $x^{-1}y \in H$, hence $x^{-1}y \notin S$, so no edge. If they are in different cosets then $x^{-1}y \notin H$, hence $x^{-1}y \in S$, so an edge exists. Therefore adjacency is exactly across distinct cosets, i.e., a complete balanced r -partite graph. ■

Proposition 2 gives the key motivation for our construction: index- r coset-based generators recover the balanced complete r -partite when all nontrivial cosets are selected, and random inverse-closed sampling from these cosets extends that structure to lower density levels. Numerically, we observe that generic random Cayley candidates are already competitive, while coset-based Cayley candidates are typi-

cally stronger.

An index- r subgroup requires that r divides n , so direct coset construction is not always available. To handle arbitrary n , we use a lift-and-delete extension: construct a Cayley graph on $n + j$ vertices where divisibility holds, then delete j vertices to return to size n . Numerically, lift-and-delete remains competitive for small lift sizes (e.g., $j = 1, 2$). The following lemma indicates that deleting one vertex is equivalent to deleting any other vertex up to graph isomorphism, which simplifies decision-making in the $j = 1$ case.

Lemma 3 (Vertex-Transitivity of Cayley Graphs): Let Γ be a finite group and $S \subseteq \Gamma \setminus \{e\}$ with $S = S^{-1}$. Then $\text{Cay}(\Gamma, S)$ is vertex-transitive.

Proof: This is a standard property of Cayley graphs; see Theorem 3.1.2 in [30]. ■

For $j = 1$, deletion is choice-free; for $j > 1$, deletion is not choice-free, so we use random deletion and leave principled selection for future work.

How the choice of group family and subgroup affects algebraic connectivity has not been systematically investigated and is left as a future direction. For implementation simplicity, we use standard group families (e.g., dihedral or cyclic) and fixed subgroup templates. We therefore define the Cayley backbone as a coset-based generating-set Cayley construction, with lift-and-delete applied when divisibility constraints prevent direct index- r construction. Algorithm 2 gives the corresponding fixed-index- r backbone procedure. When the feasible degree is too small (e.g., $k = 1$ or $k = 2$), sampled Cayley candidates can be disconnected.

Algorithm 2 Index- r Cayley Backbone Construction

- 1: **Input:** n, m, r , sample count $t \geq 1$
 - 2: **Output:** Cayley backbone G_{cay}
 - 3: compute lift-delete j , set $n_{\text{lift}} \leftarrow n + j$, $k \leftarrow \lfloor \frac{2m}{n-j} \rfloor$
 - 4: choose preset group Γ and index- r subgroup H (with lift if needed)
 - 5: set $k \leftarrow \max\{\tilde{k} \leq k : \tilde{k} \text{ is inverse-closed feasible}\}$
 - 6: set $\hat{\tau} \leftarrow \frac{n_{\text{lift}}^2}{2} \cdot \frac{r-1}{r}$, $\hat{m} \leftarrow m + jk$
 - 7: **if** $j = 0$ and $m = \hat{\tau}$ **then**
 - 8: return balanced complete r -partite graph
 - 9: **end if**
 - 10: initialize candidate set $\mathcal{C} \leftarrow \emptyset$
 - 11: **if** $\hat{m} \leq \hat{\tau}$ **then**
 - 12: sample t inverse-closed generating sets $S_1, \dots, S_t$ of size k from $\Gamma \setminus H$
 - 13: **else**
 - 14: sample t inverse-closed generating sets by taking all elements in $\Gamma \setminus H$ and filling the remaining $k - |\Gamma \setminus H|$ generators from $H \setminus \{e\}$
 - 15: **end if**
 - 16: for each sampled S_s , build $G_s \leftarrow \text{Cay}(\Gamma, S_s)$, delete j vertices, and add candidate $\tilde{G}_s$ to $\mathcal{C}$
 - 17: return $G_{\text{cay}} = \arg \max_{G \in \mathcal{C}} \lambda_2(G)$
-

At the coverage level, index- r Cayley construction reaches

edge ratios up to approximately $(r - 1)/r$. Hence indices 2, 3, 4 cover low-to-intermediate densities up to approximately 3/4, i.e., most regimes where the envelope points are not dominant in practice. Based on this coverage property, we now instantiate a deterministic density-routing rule that decides which family (or both) is evaluated at each target density, and how Cayley indices are selected under the corresponding edge feasibility.

C. Density Routing and Backbone Selection Rule

We operationalize the above family-level observations using a compact overlap rule: envelope is used for $\rho \in [0, 0.1] \cup [0.5, 1]$, Cayley is used for $\rho \in [0.05, 0.75]$, and both families are evaluated on the overlap windows $[0.05, 0.1]$ and $[0.5, 0.75]$. For Cayley candidates, we use index-2/3/4 coset constructions with inverse-closed sampling under edge-budget feasibility. Specifically, index-2 uses dihedral/semidirect templates, while indices 3 and 4 use cyclic templates. For each index $r \in \{2, 3, 4\}$, let j_r be the lift-delete count. We define $k_{\max}^{(r)}$ as the largest budget-feasible Cayley degree after lift-delete adjustment, $m_{\text{post}}^{(r)}$ as the corresponding post-delete edge count, and $d_{\text{post}}^{(r)}$ as the density-based pocket coordinate: for simplicity in this pocket computation, we assume the deleted vertices have no edges among themselves.

$$k_{\max}^{(r)} = \left\lfloor \frac{2m}{n - j_r} \right\rfloor, \quad m_{\text{post}}^{(r)} = \left\lfloor \frac{k_{\max}^{(r)}(n - j_r)}{2} \right\rfloor, \quad (7)$$

$$d_{\text{post}}^{(r)} = \frac{2m_{\text{post}}^{(r)}}{n^2}.$$

$d_{\text{post}}^{(r)}$ is the post-delete average-degree-to- n ratio; at balanced index- r endpoints it equals $1 - \frac{1}{r}$, i.e., 1/2, 2/3, 3/4 for $r = 2, 3, 4$. Density-based pocket tests are then applied as $d_{\text{post}}^{(2)} \in (0, 0.6)$, $d_{\text{post}}^{(3)} \in (\frac{1}{2}, \frac{2}{3})$, and $d_{\text{post}}^{(4)} \in (\frac{2}{3}, \frac{3}{4})$, with the smallest valid index selected when multiple are valid. This matches the practical coverage of index-2/3/4 Cayley constructions at approximately 1/2, 2/3, 3/4, respectively (exact in no-lift cases). Empirically, for $\rho > 3/4$, envelope points are typically stronger (see Figures 5 and 6), so routing favors envelope there. In overlap windows, both families are evaluated; each attempts to produce a backbone, and when both are feasible, the final family choice is made after completion by comparing final λ_2 . In Cayley-only windows, if sampled Cayley candidates are infeasible/disconnected (typically at very small feasible degree), envelope is used as fallback.

The routing rule specifies *when* each family is activated; Algorithm 3 summarizes *how* the backbone stage combines envelope construction (Algorithm 1), Cayley construction (Algorithm 2), overlap handling, and fallback.

D. Backbone Complexity and Scalability

For fixed (n, m) , envelope-level lookup is $O(\log n)$ via monotone integer binary search. In the Cayley branch, t sampled candidates are independent across samples, so construction/evaluation is sample-wise parallelizable. For pure

Cayley candidates (before deletion), λ_2 can be evaluated via character/representation decompositions [31]: $O(n \log n)$ in cyclic/abelian FFT settings and $O(nk)$ in direct dihedral/irrep summation ($k = |S|$). By contrast, FVG-style completion is path-dependent and sequential, and generic dense spectral evaluation is $O(n^3)$ naively (or $O(n^\omega)$ theoretically with $\omega \approx 2.37$ [32]).

Because backbone budgets are $O(n)$ -level discretizations of a $\Theta(n^2)$ feasible edge range, the residual gap to target is typically $\Theta(n)$ edges. Hence, when completion is the dominant sequential stage, backbone initialization can reduce the effective horizon by about one order ($\Theta(n^2) \rightarrow \Theta(n)$), consistent with observed scaling.

In practice, we use fixed $t = 500$. Increasing t from 500 up to 10^4 gives marginal quality gains in tested ranges, and runtime becomes dominated by per-candidate λ_2 evaluation rather than candidate construction. Although required t may increase with n in principle, fixed- t quality remains stable in our tested ranges.

Algorithm 3 Density-Dependent Backbone Construction

```

1: Input:  $n, m$ , Cayley sampling count  $t$ 
2: Output: Backbone candidate list  $\mathcal{B}$ 
3: compute  $\rho$  from  $(n, m)$  via (2)
4: set  $\text{env\_active} \leftarrow [\rho \in [0, 0.1] \cup [0.5, 1]]$ ,
    $\text{cay\_active} \leftarrow [\rho \in [0.05, 0.75]]$ 
5: initialize  $\mathcal{B} \leftarrow []$ 
6: if  $\text{env\_active}$  then
7:    $G_{\text{env}} \leftarrow \text{Algorithm 1}(n, m)$ 
8:   append  $G_{\text{env}}$  to  $\mathcal{B}$ 
9: end if
10: if  $\text{cay\_active}$  then
11:   select index  $r$  by (7), and run Algorithm 2( $n, m, r, t$ )
12:   if a connected Cayley candidate exists then
13:     append  $G_{\text{cay}}$  to  $\mathcal{B}$ 
14:   end if
15: end if
16: if  $\mathcal{B}$  is empty then
17:    $G_{\text{env}} \leftarrow \text{Algorithm 1}(n, m)$  ▷ fallback
18:   append  $G_{\text{env}}$  to  $\mathcal{B}$ 
19: end if
20: return  $\mathcal{B}$ 

```

III. REINFORCEMENT LEARNING COMPLETION

After backbone construction, we solve only the residual edge-allocation problem by a sequential policy. Given an initial connected backbone $G_0 = (V, E_0)$ with $|E_0| \leq m_{\text{target}}$, each step adds one non-edge for $T = m_{\text{target}} - |E_0|$ steps. Thus the RL stage does not design a graph from scratch; it refines a density-guided backbone under a fixed residual budget.

A. Residual Edge-Addition MDP

A state is the current graph $G_t = (V, E_t)$, and an action is a non-edge $(i, j) \notin E_t$. The transition is the update $E_{t+1} = E_t \cup \{(i, j)\}$, and the episode terminates at $t =$

T (equivalently $|E_t| = m_{\text{target}}$). In our implementation, episodes are sampled over a curriculum of graph sizes and random target densities, then mapped to connected-range edge budgets by (2).

B. State and Candidate Representation

At each step, we first form a candidate action pool by keeping only the top- k non-edges ranked by the Fiedler score $(\phi_2(i) - \phi_2(j))^2$ ($k = 128$), where ϕ_2, ϕ_3 are eigenvectors of λ_2, λ_3 . For each candidate, the policy uses node, pair, and global features:

$$\begin{aligned} \text{node: } & [d_i/(n-1), d_i^{(2)}/(n-1), \phi_2(i), \phi_3(i), C_i], \\ \text{pair: } & \left[(\phi_2(i) - \phi_2(j))^2, |\phi_3(i) - \phi_3(j)|, \right. \\ & \left. \text{CN}^{\text{norm}}, \text{Jaccard}, \min\{\text{dist}(i, j), d_{\text{cap}}\}/d_{\text{cap}} \right], \\ \text{global: } & [n, \rho_{\text{target}}, \rho_t, \lambda_2/n, \lambda_3/n]. \end{aligned}$$

Here d_i is the node degree, $d_i^{(2)}$ is the corresponding two-hop neighbor count (excluding one-hop neighbors), C_i is the local clustering coefficient,

$$\text{CN}^{\text{norm}} = \frac{|N(i) \cap N(j)|}{n-2}, \quad \text{Jaccard} = \frac{|N(i) \cap N(j)|}{|N(i) \cup N(j)|},$$

$\text{dist}(i, j)$ is shortest-path distance in G_t , and d_{cap} is a fixed distance cap used to truncate distance features and control computation cost.

C. Policy and Value Networks

Let x_i denote node features and h_i denote the corresponding graph attention network (GAT) embedding. For each candidate non-edge (i, j) , a symmetric edge representation is built from

$$|h_i - h_j|, \quad h_i \odot h_j, \quad |x_i - x_j|, \quad x_i \odot x_j,$$

then concatenated with pair and global features and scored by a multi-layer perceptron (MLP). The action distribution is the softmax over candidate scores. A value head shares the GAT trunk and predicts $V(s_t)$ from pooled node embedding plus global features.

D. Training Objective

Completion is trained with PPO, using a value baseline and entropy regularization. The per-step reward is

$$r_t = \frac{\lambda_2(G_{t+1}) - \lambda_2(G_t)}{n} + \beta \cdot \mathbf{1}_{\text{terminal}} \frac{\lambda_2(G_T)}{n},$$

where β is the terminal-bonus coefficient. Returns are Monte Carlo discounted sums,

$$A_t = \mathcal{R}_t - V_\psi(s_t), \quad \mathcal{R}_t = \sum_{\tau=t}^{T-1} \gamma^{\tau-t} r_\tau.$$

With importance ratio $\eta_t(\theta) = \pi_\theta(a_t|s_t)/\pi_{\theta_{\text{old}}}(a_t|s_t)$ and clipped ratio $\bar{\eta}_t(\theta) = \text{clip}(\eta_t(\theta), 1 - \epsilon, 1 + \epsilon)$, we use:

$$\begin{aligned} \mathcal{L}_{\text{PPO}} = & -\mathbb{E}[\min(\eta_t(\theta)A_t, \bar{\eta}_t(\theta)A_t)] \\ & + c_v \mathbb{E}[(V_\psi(s_t) - \mathcal{R}_t)^2] - c_e \mathbb{E}[\mathcal{H}(\pi_\theta(\cdot|s_t))]. \end{aligned}$$

PPO is trained for 1.2×10^6 environment steps using a three-phase size curriculum, with 4×10^5 steps per phase over $n \in [12, 16]$, $n \in [16, 32]$, and $n \in [16, 64]$.

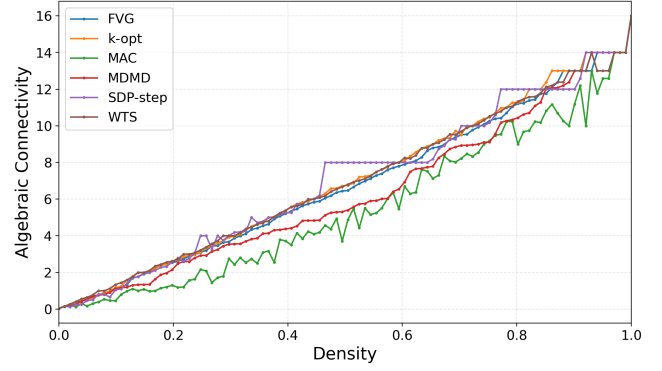


Fig. 3. Baseline quality at $n = 16$.

IV. NUMERICAL RESULTS

A. Experimental Setup

All methods are evaluated under the same edge-budget mapping (2). For each tested n , we sweep 100 density points over $\rho \in [0, 1]$, and each point maps to the same (n, m) target across methods. Each (n, ρ) point is repeated 5 times; tables report mean/std over the density sweep from these per-point repeats. The quality metric is λ_2 at the target budget, and runtime is wall-clock time. Experiments are conducted on the Seawulf HPC cluster (Stony Brook University), long-40core partition, in exclusive-node mode using 1 node (8 CPU cores, 64 GB RAM) per run. In all experiments, both sample-wise Cayley parallelization and character-based spectral acceleration are disabled. Most reported n are powers of two for clarity of scaling, but additional odd/even/prime/random n show similar qualitative behavior in our experiments; a finer characterization of how performance varies with integer structure of n is left for future work. Implementation details, hyperparameter settings, and experiment scripts are available in our public repository (GitHub).

B. Baseline Comparison Among Existing Methods

This subsection benchmarks full-construction baselines. Compared methods include constructive heuristics (FVG [19], MDMD [25]), local search (WTS [23], k-opt [20]), relaxation/optimization (MAC [24], SDP-step [23]), and exact OA-PMC [18]. Tables I–II summarize quality/runtime, and Figures 3–4 show representative quality curves.

Results show the expected trade-off: MAC/MDMD are fastest but weaker; WTS/k-opt are usually slower and only occasionally better at small n ; SDP-step/OA-PMC are stronger when applicable but much more expensive. Hence FVG is the primary practical full-construction baseline.

C. Backbone-Only Performance Across Density Regimes

We compare backbone-only outputs (no RL completion), including EIG [15], a complete-multipartite constructor based on complement-component decomposition that may

TABLE I

QUALITY SUMMARY. FVG: ABSOLUTE λ_2 (MEAN $\pm$ STD). OTHERS: $\Delta\lambda_2$ MEAN $\pm$ STD / WIN% / MATCH%. **RED** = BEST $\Delta\lambda_2$; **BLUE** = BEST WIN%.

Method	$n = 8$	$n = 16$	$n = 32$	$n = 64$	$n = 128$
FVG	3.34 ± 1.77	6.72 ± 4.18	13.99 ± 8.81	28.87 ± 18.19	59.16 ± 36.84
MAC	$-0.47 \pm 0.41/0/25$	$-1.27 \pm 0.65/1/2$	$-3.03 \pm 1.42/0/2$	$-6.61 \pm 3.11/0/1$	$-13.50 \pm 6.33/0/1$
MDMD	$-0.08 \pm 0.35/10/50$	$-0.47 \pm 0.40/7/12$	$-1.09 \pm 0.73/4/5$	$-2.15 \pm 1.49/1/3$	$-3.89 \pm 2.49/1/1$
WTS	$0.27 \pm 0.36/50/50$	$0.18 \pm 0.22/83/12$	$-0.23 \pm 0.31/12/2$	$-0.76 \pm 0.63/2/2$	$-1.63 \pm 1.12/1/1$
k-opt	$0.19 \pm 0.29/45/55$	$0.20 \pm 0.16/81/19$	$0.13 \pm 0.13/90/10$	$0.04 \pm 0.05/80/20$	$0.01 \pm 0.02/58/8$
SDP-step	$0.08 \pm 0.57/30/40$	$0.29 \pm 0.57/67/11$	$0.39 \pm 0.71/71/4$	—	—
OA-PMC	$0.31 \pm 0.40/55/45$	—	—	—	—
BB-RL(Ours)	$0.28 \pm 0.46/35/55$	$0.38 \pm 0.54/61/14$	$0.59 \pm 0.68/68/5$	$0.87 \pm 0.97/71/3$	$1.34 \pm 1.80/65/2$

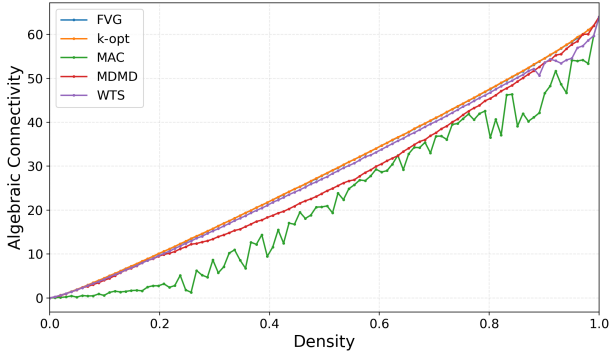
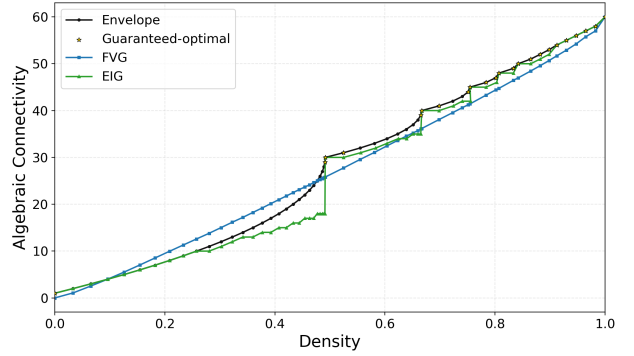
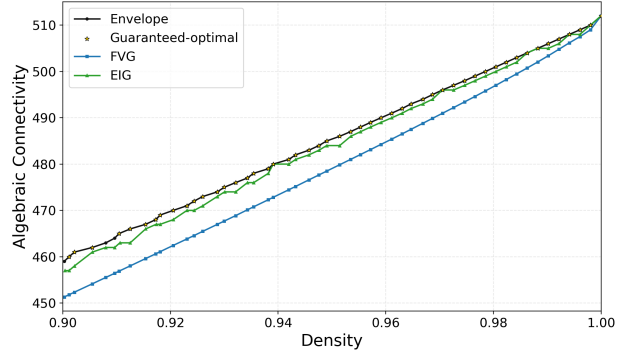
Fig. 4. Baseline quality at $n = 64$.Fig. 5. Envelope, FVG, and EIG at $n = 60$.

TABLE II

RUNTIME SUMMARY (MEAN SECONDS), WITH BB-RL DECOMPOSITION.

RED = FASTEST METHOD AT EACH n .

Method	$n = 8$	$n = 16$	$n = 32$	$n = 64$	$n = 128$
MAC	3.07×10^{-3}	4.65×10^{-3}	1.95×10^{-2}	6.06×10^{-2}	2.10×10^{-1}
MDMD	8.59×10^{-4}	5.42×10^{-3}	4.92×10^{-2}	4.08×10^{-1}	5.14
FVG	1.38×10^{-3}	7.46×10^{-3}	1.02×10^{-1}	9.27×10^{-1}	24.06
WTS	5.47×10^{-1}	1.00	2.14	5.53	19.78
k-opt	5.20×10^{-2}	4.04×10^{-1}	2.70	8.60	38.78
SDP-step	1.48×10^{-1}	1.42	24.88	—	—
OA-PMC	10.62	—	—	—	—
BB-RL(Ours, total)	5.72×10^{-3}	1.10×10^{-2}	2.74×10^{-2}	1.22×10^{-1}	7.68×10^{-1}
init	3.07×10^{-3}	3.86×10^{-3}	9.05×10^{-3}	5.56×10^{-2}	3.14×10^{-1}
completion	2.72×10^{-3}	7.74×10^{-3}	1.94×10^{-2}	7.44×10^{-2}	4.95×10^{-1}

Fig. 6. Envelope, FVG, and EIG at $n = 512$, $\rho > 0.9$.

return $m \geq m_{\text{target}}$. Figures 5–7 show dense and low-to-intermediate regimes; the envelope optimality-certified subset follows Corollary 4.6 of [15]. Overall, envelope dominates dense regimes, while Cayley is competitive in low-to-intermediate regimes.

D. Backbone+RL Performance

We compare backbone-initialized RL and FVG on identical (n, m) targets; Figure 8 and Tables I–II summarize quality and runtime. Across tested densities and n values, backbone+RL is the only non-exact tested method that avoids the usual quality–scalability trade-off as n grows. Reported runtimes are serial, and Cayley sampling admits straightforward parallel speedup, unlike sequential FVG.

V. CONCLUSION

This paper presented a density-dependent framework for fixed- (n, m) algebraic-connectivity maximization that separates structural initialization from completion. A high-quality low-cost backbone plus RL completion gives the best non-exact quality/runtime scaling among tested baselines as n increases. As n grows, cheaper heuristics lose quality while stronger optimization-based methods become increasingly expensive, supporting backbone-first completion as a practical direction for scalability. This suggests viewing algebraic-connectivity maximization as a structural family-design stage plus a residual completion stage. Open directions include identifying other cheaply constructible (near-)optimal graph families to densify backbone coverage,

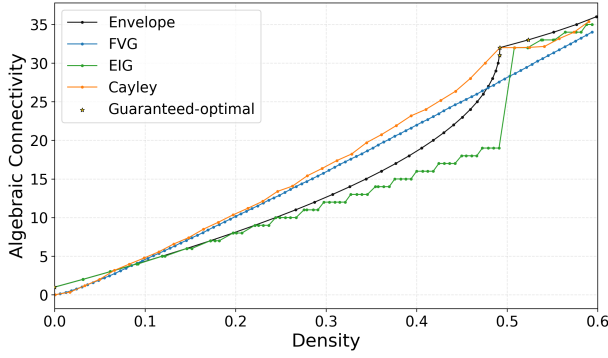


Fig. 7. Low-intermediate quality at $n = 64$.

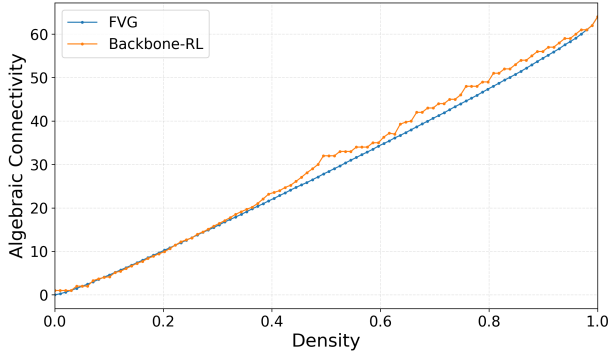


Fig. 8. Backbone+RL vs FVG at $n = 64$.

principled group/subgroup selection for Cayley backbones, characterizing how density-dependent behavior varies with integer structure of n , and lightweight density-aware completion policies.

REFERENCES

- [1] A. Jadbabaie, J. Lin, and A.S. Morse. Coordination of groups of mobile autonomous agents using nearest neighbor rules. *IEEE Transactions on Automatic Control*, 48(6):988–1001, 2003.
- [2] R. Olfati-Saber and R.M. Murray. Consensus problems in networks of agents with switching topology and time-delays. *IEEE Transactions on Automatic Control*, 49(9):1520–1533, 2004.
- [3] W. Ren and R.W. Beard. Consensus seeking in multiagent systems under dynamically changing interaction topologies. *IEEE Transactions on Automatic Control*, 50(5):655–661, 2005.
- [4] R. Olfati-Saber, J.A. Fax, and R.M. Murray. Consensus and cooperation in networked multi-agent systems. *Proceedings of the IEEE*, 95(1):215–233, 2007.
- [5] A. Nedić and A. Ozdaglar. Distributed subgradient methods for multi-agent optimization. *IEEE Transactions on Automatic Control*, 54(1):48–61, 2009.
- [6] S. Mou, J. Liu, and A.S. Morse. A distributed algorithm for solving a linear algebraic equation. *IEEE Transactions on Automatic Control*, 60(11):2863–2878, 2015.
- [7] L. Wang, J. Liu, B.D.O. Anderson, and A.S. Morse. Split-spectrum based distributed state estimation for linear systems. *Automatica*, 161:111421, 2024.
- [8] M. Fiedler. Algebraic connectivity of graphs. *Czechoslovak Mathematical Journal*, 23(2):298–305, 1973.
- [9] S. Kar and J.M.F. Moura. Topology for global average consensus. In *Proceedings of the 40th Asilomar Conference on Signals, Systems and Computers*, pages 276–280, 2006.
- [10] M. Cao and C.W. Wu. Topology design for fast convergence of network consensus algorithms. In *Proceedings of the 2007 IEEE International Symposium on Circuits and Systems*, pages 1029–1032, 2007.
- [11] M. Rafiee and A.M. Bayen. Optimal network topology design in multi-agent systems for efficient average consensus. In *Proceedings of the 49th IEEE Conference on Decision and Control*, pages 3877–3883, 2010.
- [12] R. Dai and M. Mesbahi. Optimal topology design for dynamic networks. In *Proceedings of the 50th IEEE Conference on Decision and Control*, pages 1280–1285, 2011.
- [13] D. Xue, A. Gusrialdi, and S. Hirche. A distributed strategy for near-optimal network topology design. In *Proceedings of the 21st International Symposium on Mathematical Theory of Networks and Systems*, pages 7–14, 2014.
- [14] K. Ogiwara, T. Fukami, and N. Takahashi. Maximizing algebraic connectivity in the space of graphs with a fixed number of vertices and edges. *IEEE Transactions on Control of Network Systems*, 4(2):359–368, 2017.
- [15] K. Shahbaz, M.N. Belur, and A. Ganesh. Algebraic connectivity: local and global maximizer graphs. *IEEE Transactions on Network Science and Engineering*, 10(3):1636–1647, 2023.
- [16] D. Mosk-Aoyama. Maximum algebraic connectivity augmentation is NP-hard. *Operations Research Letters*, 36(6):677–679, 2008.
- [17] M. Kurahashi, N. Salaani, T. Migita, and N. Takahashi. Depth-first search algorithms for algebraic connectivity maximizing regular graphs. In *Proceedings of the 12th International Symposium on Computing and Networking*, pages 156–161, 2024.
- [18] N. Somisetty, H. Nagarajan, and S. Darbha. Optimal robust network design: formulations and algorithms for maximizing algebraic connectivity. *IEEE Transactions on Control of Network Systems*, 12(1):918–929, 2025.
- [19] A. Ghosh and S. Boyd. Growing well-connected graphs. In *Proceedings of the 45th IEEE Conference on Decision and Control*, pages 6605–6611, 2006.
- [20] N. Somisetty, H. Nagarajan, and S. Darbha. Spectral graph theoretic methods for enhancing network robustness in robot localization. In *Proceedings of the 63rd IEEE Conference on Decision and Control*, pages 8811–8818, 2024.
- [21] M. Zhou, G. Liu, K. Lu, R. Mao, and H. Liao. Accelerating the decentralized federated learning via manipulating edges. In *Proceedings of the 2024 ACM Web Conference*, pages 2945–2954, 2024.
- [22] Y. Kim. Bisection algorithm of increasing algebraic connectivity by adding an edge. *IEEE Transactions on Automatic Control*, 55(1):170–174, 2010.
- [23] P. Wei, L. Chen, and D. Sun. Algebraic connectivity maximization of an air transportation network: the flight routes’ addition/deletion problem. *Transportation Research Part E: Logistics and Transportation Review*, 61:13–27, 2014.
- [24] K.J. Doherty, D.M. Rosen, and J.J. Leonard. Spectral measurement sparsification for pose-graph SLAM. In *Proceedings of the 2022 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 1–8, 2022.
- [25] Gang Li, Zhi Feng Hao, Han Huang, and Hang Wei. Maximizing algebraic connectivity via minimum degree and maximum distance. *IEEE Access*, 6:41249–41255, 2018.
- [26] Ryoya Ishii and Norikazu Takahashi. Extensions of a theorem on algebraic connectivity maximizing graphs. In *Proceedings of the 2016 International Symposium on Nonlinear Theory and Its Applications (NOLTA2016)*, pages 598–601, November 2016.
- [27] Steven Klee and Matthew T. Stamps. Eigenvalues of graph laplacians via rank-one perturbations. *The Quarterly Journal of Mathematics*, 73(2):609–616, 2022.
- [28] B. Mohar. The laplacian spectrum of graphs. In *Graph Theory, Combinatorics, and Applications*, pages 871–898. Wiley, New York, NY, USA, 1991.
- [29] T. Kolokolnikov. Maximizing algebraic connectivity for certain families of graphs. *Linear Algebra and its Applications*, 471:122–140, 2015.
- [30] Chris Godsil and Gordon Royle. *Algebraic Graph Theory*, volume 207 of *Graduate Texts in Mathematics*. Springer, 2001.
- [31] László Babai. Spectra of cayley graphs. *Journal of Combinatorial Theory, Series B*, 27(2):180–189, 1979.
- [32] François Le Gall. Powers of tensors and fast matrix multiplication. In *Proceedings of the 39th International Symposium on Symbolic and Algebraic Computation*, pages 296–303, 2014.